Lab Handout 4-D

Support Vector Machines with Grid Search and Permutation Importance

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Train an `SVC` classifier (with both linear and RBF kernels) for each of the four tasks, grid-search its hyperparameters over C and γ, and produce a permutation-importance analysis that gives a *model-agnostic* ranking of which features the classifier relied on. Permutation importance is trustworthy where Gini importance (from LH4-C) can be misleading for correlated features.

2 Prerequisites

- Lab Handout 4-B complete (`Code/AIML/artifacts/{sf,mod,beacon,pkt}/` with scaled feature matrices). SVMs *require* scaling — do not attempt to train on unscaled features.
- Lab Handout 4-C complete (`baselines_manifest.json` and per-task `rf.pkl` available for comparison in Step 4).

3 Background: Why Both Kernels?

The linear kernel gives you interpretable coefficients (one per feature per one-vs-rest binary problem) but can only learn linearly separable decision surfaces. The RBF kernel handles non-linear class boundaries at the cost of interpretability and $O(n^2)$ training time. Comparing the two tells you whether the classes are linearly separable in the chosen feature space:

- **Linear $\approx$ RBF accuracy:** the feature space is (nearly) linearly separable; a linear model is the right tool.
- **RBF $\gg$ Linear:** the decision boundary is curved in this feature space; expect a neural network (LH4-E) to also do well.
- **Linear $>$ RBF:** the RBF is overfitting on noise (rare, usually indicates a bad γ).

4 Procedure

4.1 Step 1 — Logging, loader, and grid search

The script (`Code/AIML/svm_train.py`) follows the same logging pattern as LH4-A through LH4-C: stdout plus a timestamped archive log and `svm_train_latest.log` in `Code/AIML/Logs/`. Single-class tasks (`mod` in the current LoRa-only dataset) are skipped because `SVC.fit` requires ≥ 2 classes.

```python
import json
import logging
import sys
import time
from datetime import datetime, timezone
from math import sqrt
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm, spearmanr
from sklearn.inspection import permutation_importance
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
                             classification_report, confusion_matrix)
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

SCRIPT_DIR    = Path(__file__).parent
LOG_DIR       = SCRIPT_DIR / "Logs"
ARTIFACTS_DIR = SCRIPT_DIR / "artifacts"
FIGS_DIR      = SCRIPT_DIR / "figs"
MANIFEST_PATH = SCRIPT_DIR / "svm_manifest.json"

LOG_DIR.mkdir(parents=True, exist_ok=True)
FIGS_DIR.mkdir(parents=True, exist_ok=True)

run_stamp   = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
log_archive = LOG_DIR / f"svm_train_{run_stamp}.log"
log_latest  = LOG_DIR / "svm_train_latest.log"

log = logging.getLogger("svm_train")
log.setLevel(logging.DEBUG)
log.handlers.clear()
fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s",
                        datefmt="%Y-%m-%d %H:%M:%S")
for h in (logging.StreamHandler(sys.stdout),
          logging.FileHandler(log_archive, mode="w", encoding="utf-8"),
          logging.FileHandler(log_latest,  mode="w", encoding="utf-8")):
    h.setFormatter(fmt)
    log.addHandler(h)

log.info("=== svm_train.py starting ===")
log.info(f"Run stamp: {run_stamp}")


def load_task(task):
    d = ARTIFACTS_DIR / task
    X_tr = np.load(d / "X_train.npy"); y_tr = np.load(d / "y_train.npy")
    X_va = np.load(d / "X_val.npy");   y_va = np.load(d / "y_val.npy")
    X_te = np.load(d / "X_test.npy");  y_te = np.load(d / "y_test.npy")
    pre  = joblib.load(d / "preprocessor.pkl")
    return X_tr, y_tr, X_va, y_va, X_te, y_te, pre
```

```python


def wilson_ci(n_correct, n_total, alpha=0.05):
    if n_total == 0:
        return 0.0, 1.0
    p = n_correct / n_total
    z = norm.ppf(1 - alpha / 2)
    denom = 1 + z * z / n_total
    center = (p + z * z / (2 * n_total)) / denom
    half = z * sqrt(p * (1 - p) / n_total
                    + z * z / (4 * n_total * n_total)) / denom
    return max(0.0, center - half), min(1.0, center + half)
```

Listing 1: Top of `svm_train.py` — imports, paths, logging, loader, Wilson CI.

```python
SVM_GRID = [
    {"kernel": ["linear"], "C": [0.1, 1, 10]},
    {"kernel": ["rbf"],    "C": [0.1, 1, 10, 100],
                           "gamma": ["scale", 0.01, 0.1, 1.0]},
]

results = {}
for task in ["sf", "mod", "beacon", "pkt"]:
    log.info(f"--- Task: {task} ---")
    X_tr, y_tr, X_va, y_va, X_te, y_te, pre = load_task(task)
    n_classes = len(np.unique(y_tr))
    log.info(f"  Train shape={X_tr.shape}, classes={n_classes}, "
             f"features={pre['feature_cols']}")
    if n_classes < 2:
        log.warning(f"  {task}: only {n_classes} class --- skipping (SVC needs >=2)")
        results[task] = {"skipped": True, "reason": "single class in train"}
        continue

    X_cv = np.concatenate([X_tr, X_va]); y_cv = np.concatenate([y_tr, y_va])
    t0 = time.time()
    gs = GridSearchCV(SVC(random_state=42),
                      SVM_GRID, cv=5, scoring="accuracy",
                      n_jobs=-1, verbose=0)
    gs.fit(X_cv, y_cv)
    elapsed = time.time() - t0
    log.info(f"  Grid search complete in {elapsed:.1f}s; "
             f"best params: {gs.best_params_}")

    best = gs.best_estimator_
    y_pred = best.predict(X_te)
    n_correct = int((y_pred == y_te).sum())
    acc_te = n_correct / len(y_te)
    lo, hi = wilson_ci(n_correct, len(y_te))
    log.info(f"  CV acc = {gs.best_score_:.3f}  test acc = {acc_te:.3f}  "
             f"[95% CI {lo:.3f}, {hi:.3f}]")

```

```python
    joblib.dump({"model": best, "cv_acc": float(gs.best_score_),
                 "test_acc": acc_te, "ci": [lo, hi],
                 "params": gs.best_params_, "elapsed_s": elapsed},
                ARTIFACTS_DIR / task / "svm.pkl")
    results[task] = {
        "kernel":   gs.best_params_.get("kernel"),
        "C":        gs.best_params_.get("C"),
        "gamma":    gs.best_params_.get("gamma"),
        "cv_acc":   float(gs.best_score_),
        "test_acc": acc_te, "ci_lo": lo, "ci_hi": hi,
        "n_correct": n_correct, "n_total": len(y_te),
        "elapsed_s": elapsed,
        "feature_cols": pre["feature_cols"],
    }
```

Listing 2: SVM grid search per task with single-class skip and elapsed-time logging.

SVM Training Time

At $n \approx 2{,}000$ samples with the full grid (linear: 3 combos; RBF: 16 combos; both with 5-fold CV), expect roughly 30 s–3 min per task on a laptop with `n_jobs=-1`. If one combination hangs for >30 minutes, interrupt it — $\gamma = 1.0$ with $C = 100$ on a scaled feature set can produce an extremely slow fit due to a near-singular kernel matrix. Remove that corner from the grid and rerun. The script logs per-task elapsed time so this is easy to spot.

4.2 Step 2 — Classification report and confusion matrix

```python
for task, res in results.items():
    if res.get("skipped"):
        continue
    pkl = joblib.load(ARTIFACTS_DIR / task / "svm.pkl")
    svm = pkl["model"]
    _, _, _, _, X_te, y_te, pre = load_task(task)
    y_pred = svm.predict(X_te)
    log.info(f"--- {task} SVM classification report ---")
    report = classification_report(y_te, y_pred,
        target_names=[str(c) for c in pre["encoder"].classes_])
    for line in report.splitlines():
        log.info(line)

    cm = confusion_matrix(y_te, y_pred)
    fig, ax = plt.subplots(figsize=(5, 4))
    ConfusionMatrixDisplay(cm,
        display_labels=[str(c) for c in pre["encoder"].classes_]
    ).plot(cmap="Blues", values_format="d", ax=ax, colorbar=False)
    ax.set_title(f"Confusion matrix --- SVM --- {task}")
    fig.tight_layout()
    out = FIGS_DIR / f"cm_svm_{task}.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    log.info(f"  Saved {out}")
```

Look at off-diagonal structure: a confusion matrix where errors cluster on SF7↔SF8 (adjacent spreading factors) is physically expected; a matrix where SF7↔SF10 dominates is not.

4.3 Step 3 — Permutation importance

Gini importance from a forest can attribute all the signal to whichever of two correlated features the tree split on first. Permutation importance is model-agnostic and directly measures "how much does test accuracy drop if I shuffle this column?"

```
def perm_importance(model, X_te, y_te, feature_names, task, n_repeats=20):
    r = permutation_importance(model, X_te, y_te,
                               n_repeats=n_repeats, random_state=42,
                               n_jobs=-1)
    order = np.argsort(r.importances_mean)[::-1]
    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.boxplot(r.importances[order].T,
               tick_labels=[feature_names[i] for i in order])
    plt.setp(ax.get_xticklabels(), rotation=30, ha="right")
    ax.set_ylabel("accuracy drop when shuffled")
    ax.set_title(f"Permutation importance --- SVM --- {task}")
    fig.tight_layout()
    out = FIGS_DIR / f"perm_importance_svm_{task}.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    log.info(f"  Saved {out}")
    ranking = []
    for i in order:
        log.info(f"    {feature_names[i]:20s}  "
                 f"mean={r.importances_mean[i]:+.3f}  "
                 f"std={r.importances_std[i]:.3f}")
        ranking.append((feature_names[i], float(r.importances_mean[i]),
                        float(r.importances_std[i])))
    return ranking


for task, res in results.items():
    if res.get("skipped"):
        continue
    pkl = joblib.load(ARTIFACTS_DIR / task / "svm.pkl")
    _, _, _, _, X_te, y_te, pre = load_task(task)
    log.info(f"--- {task} permutation importance (SVM) ---")
    res["perm_importance"] = perm_importance(
        pkl["model"], X_te, y_te, pre["feature_cols"], task)
```

4.4 Step 4 — Cross-check against random forest

Rank the features by permutation importance (SVM, Step 3) and by Gini importance (RF, from `baselines_manifest.json`). Compute Spearman rank correlation between the two rankings: $\rho > 0.8$ is strong agreement, $\rho < 0.5$ is informative disagreement (typically caused by correlated features that the two models split differently).

```
baseline_manifest = SCRIPT_DIR / "baselines_manifest.json"
if baseline_manifest.exists():
```

```
    with open(baseline_manifest) as f:
        rf_manifest = json.load(f)["results"]
else:
    log.warning("baselines_manifest.json not found --- "
                "skipping RF cross-check (run LH4-C first)")
    rf_manifest = {}

for task, res in results.items():
    if res.get("skipped") or task not in rf_manifest \
            or rf_manifest[task].get("skipped"):
        continue
    rf_rank = [(name, gini) for (name, gini)
               in rf_manifest[task]["feature_importance"]]
    svm_rank = res.get("perm_importance", [])
    common = [c for c, _, _ in svm_rank
              if c in dict(rf_rank)]
    if len(common) < 2:
        log.warning(f"  {task}: too few shared features for rank correlation")
        continue
    rf_ord  = {c: r for r, (c, _) in enumerate(rf_rank)  if c in common}
    svm_ord = {c: r for r, (c, _, _) in enumerate(svm_rank) if c in common}
    rho, _ = spearmanr([rf_ord[c]  for c in common],
                       [svm_ord[c] for c in common])
    log.info(f"  {task}: RF Gini vs SVM perm rank correlation rho = {rho:+.2f}")
    res["rank_correlation_vs_rf"] = float(rho)
```

4.5 Step 5 — Head-to-head summary and manifest

```
log.info("=== RF vs SVM Summary ===")
log.info(f"{'Task':<8} {'RF acc':>8} {'SVM acc':>8} {'delta':>8} "
         f"{'kernel':>8} {'rho_imp':>8}")
for task in ["sf", "mod", "beacon", "pkt"]:
    res = results[task]
    if res.get("skipped"):
        log.info(f"{task:<8} (skipped)")
        continue
    rf_acc = (rf_manifest.get(task, {}).get("rf", {}) or {}).get("acc")
    delta = res["test_acc"] - rf_acc if rf_acc is not None else None
    rho = res.get("rank_correlation_vs_rf")
    log.info(f"{task:<8} "
             f"{rf_acc:>8.3f}" if rf_acc is not None else f"{task:<8} {'n/a':>8} "
             f"{res['test_acc']:>8.3f} "
             f"{(f'{delta:+.3f}' if delta is not None else 'n/a'):>8} "
             f"{str(res['kernel']):>8} "
             f"{(f'{rho:+.2f}' if rho is not None else 'n/a'):>8}")

manifest = {
    "generated_at": datetime.now(timezone.utc).isoformat(),
    "run_stamp":    run_stamp,
```

```
    "log_archive":  str(log_archive),
    "svm_grid":     SVM_GRID,
    "results":      results,
}
with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2, default=str)
log.info(f"Wrote {MANIFEST_PATH}")
log.info("=== svm_train.py done ===")
```

Task	RF acc	SVM acc	Δ	Kernel	Importance ρ (RF vs SVM)
SF Detection					
Modulation	(skipped — only LoRa collected)				
Emitter ID					
Packet Type					

Table 1: Classical-model head-to-head. Carry this forward into LH4-E so the neural-network results are benchmarked against the stronger of the two. Fill from the script's printed summary or from `svm_manifest.json`.

When the Linear Kernel Wins

If the linear SVM beats the RBF by more than 1 percentage point on a given task, that is a genuinely useful result: it means (a) a simpler, faster model is preferable for deployment, and (b) the problem is linearly separable in the chosen feature space. Do not over-engineer a non-linear solution where a linear one suffices — the deployment latency budget in LH4-H will thank you.

5 Verification Checklist

- ☐ SVM grid search complete for every task with ≥ 2 classes (`sf`, `beacon`, `pkt` in the current dataset; `mod` skipped with logged reason).
- ☐ Wilson 95% CI reported for each SVM test accuracy.
- ☐ Confusion matrix generated and inspected for physical plausibility.
- ☐ Permutation importance computed with $\geq$20 repeats per task.
- ☐ Spearman rank correlation between RF Gini and SVM permutation rankings logged per task.
- ☐ `Code/AIML/artifacts/<task>/svm.pkl` saved for each non-skipped task.
- ☐ `Code/AIML/svm_manifest.json` written.
- ☐ `Code/AIML/Logs/svm_train_<timestamp>.log` archived.

Lab Handout 4-D Deliverable

1. `Code/AIML/svm_train.py`: the runnable script.
2. `Code/AIML/svm_manifest.json`: per-task best params, accuracies (with Wilson CI), elapsed time, permutation importance, and rank correlation versus RF.
3. `Code/AIML/artifacts/{sf,beacon,pkt}/svm.pkl`: the saved models.
4. `Code/AIML/figs/cm_svm_<task>.png`: confusion matrix per non-skipped task.
5. `Code/AIML/figs/perm_importance_svm_<task>.png`: permutation importance per non-skipped task.
6. Filled head-to-head RF vs. SVM summary table with the winner annotated and a one-paragraph discussion of feature-importance agreement/disagreement (using the logged Spearman ρ values).
7. `Code/AIML/Logs/svm_train_<timestamp>.log`: archived audit trail.